DBT Day 7 — Conditions, Loops & Built-in Functions

Shivam Zanjurne · Thu, 15 Oct 26 · Procedure decisions (`IF` / `CASE`) · Loops (`LOOP` + `LEAVE` , `WHILE` , `REPEAT` , `ITERATE` , nested `WHILE`) · Built-in string, numeric, and date functions

1. Warm-up — DELIMITER and Parameters, Asked Back 1

Part A — Conditions: IF and CASE 4

2. Simple IF — Pass, or Silence 4

3. IF … ELSE — Even or Odd with MOD 5

4. IF / ELSEIF / ELSE — calculate_grade 6

5. CASE — Simple versus Searched 10

Part B — Loops 12

6. LOOP with LEAVE — Print 1 to 5 12

7. WHILE — Test Before the Body 13

8. REPEAT … UNTIL — Body First, Then the Test 14

9. Where the Condition Sits 15

10. ITERATE — Skip One Pass 16

11. Nested WHILE — a 2 by 3 Grid 17

Part C — Built-in Functions 20

12. Why This Part Is on the Exam 20

13. String Functions 20

14. Numeric Functions 23

15. Date and Time Functions 25

16. Day Wrap — Rapid Revision and the Assignment 26

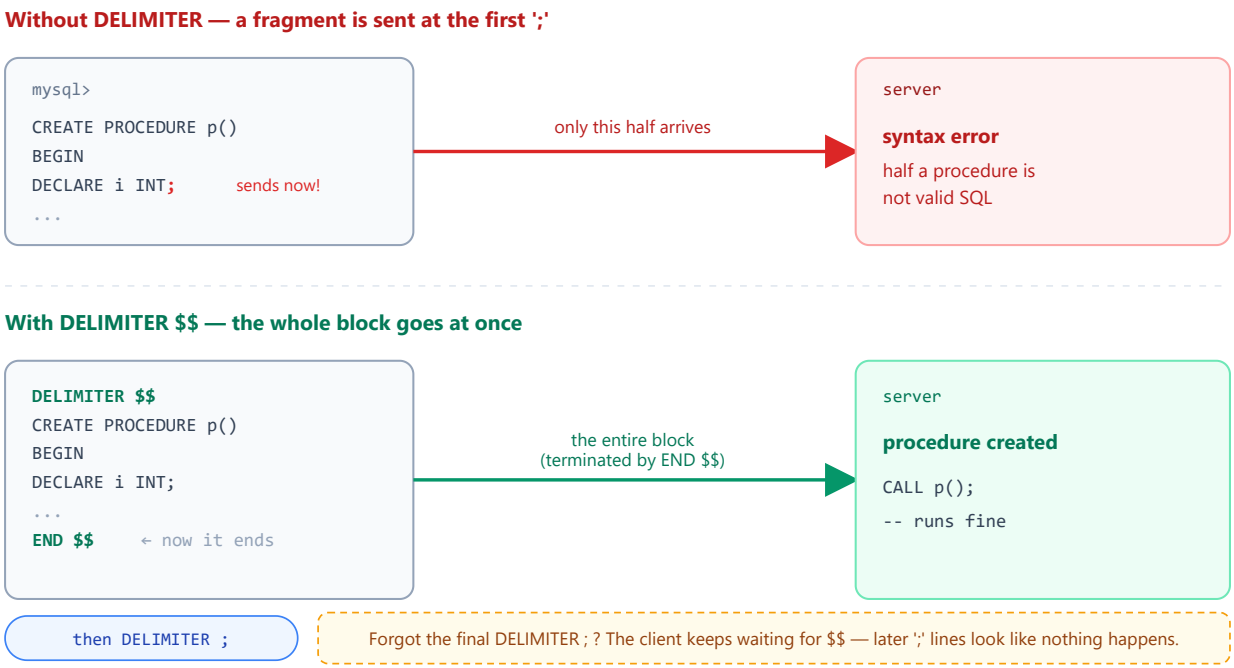
> NOTE
>
> **How to use this document.** Day 7 stays on the procedures database from Day 6 (`october_14`) and the same `student` table: `student_id` , `name` , `department_id` , `marks` . Every `sql` block is meant to be typed in MySQL CMD, in order. Part C is the MCQ block — class said to expect two or three CCEE questions from it.

1. Warm-up — DELIMITER and Parameters, Asked Back

Class did not open with a new topic. The instructor asked the room to start a procedure, then refused to move on until someone could say what `DELIMITER` actually does.

Vidya's answer is the one to keep: semicolons **appear inside** the procedure, and they must **not** be treated as the end of the statement you are sending.

`DELIMITER $$` tells the client: stop treating `;` as "send this now", and wait for `$$` . `DELIMITER ;` tells it: behave normally again.

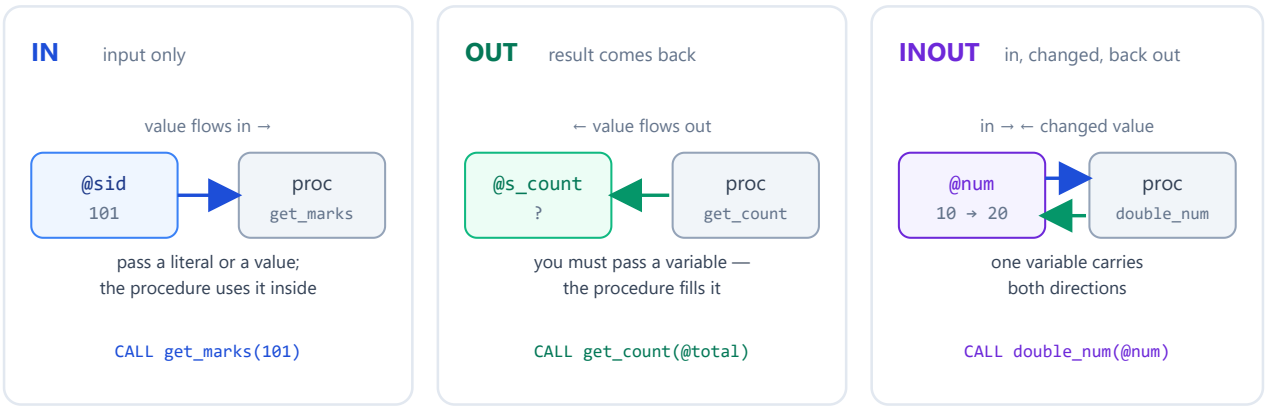


Without DELIMITER the body is cut at the first semicolon; with DELIMITER $$ the whole procedure travels at once

Rule from class	What to do
Write it twice	once **before** `CREATE PROCEDURE`, once **after** `END`
Same symbol both times	open with `$$`, close with `$$`. Mixing `$$` and `//` breaks
Safe symbols	`$$` and `//`. Do not pick a character MySQL already reserves
The restore line	`DELIMITER ;` — keyword, space, semicolon

> The long failure story (the session that had to be reopened) is Day 6, the DELIMITER section. This is the third class in a row it has come up.

IN, OUT, INOUT — the three modes, one more time



IN goes in, OUT comes back into a variable, INOUT goes in and comes back changed

Mode	Direction	What you pass at `CALL`

IN	caller → procedure	a literal or a variable, e.g. `CALL p(101)`
OUT	procedure → caller	a `@variable` that receives the answer
INOUT	both	a `@variable` that goes in and comes back changed

> NOTE
> **Q (asked in class):** "If there are multiple parameters, do we write `IN` every time?" — `IN` is the **default mode**, so it may be omitted on an input parameter. Every `OUT` and `INOUT` still needs its **own** keyword. Modes do not carry forward to the next parameter.

Today's new procedures mostly take one `IN` value and `SELECT` a result set back. The decisions and the loops are the new material.

Three parts, in the order class taught them

Part	Question it answers
A — Conditions	Given marks or a department id, which result should print?
B — Loops	How do you repeat a statement, stop it, or skip one pass?
C — Built-in functions	Which function returns this string, number, or date?

Part C is the one class flagged for the exam. Parts A and B are the procedures you are expected to write, not just recognise.

Part A — Conditions: IF and CASE

In this part: a one-way `IF` · `IF … ELSE` · the `calculate_grade` ladder · simple `CASE` versus searched `CASE`.

A procedure that only runs straight through is a script. Conditions are what make it decide. Every block you open has its own closer: **`END IF`** for an if, **`END CASE`** for a case, and **`END`** for the procedure. Those are different words.

2. Simple IF — Pass, or Silence

The smallest decision is: if this is true, run these statements. If it is false, do nothing.

```
IF <condition> THEN
    <statements>
END IF;
```

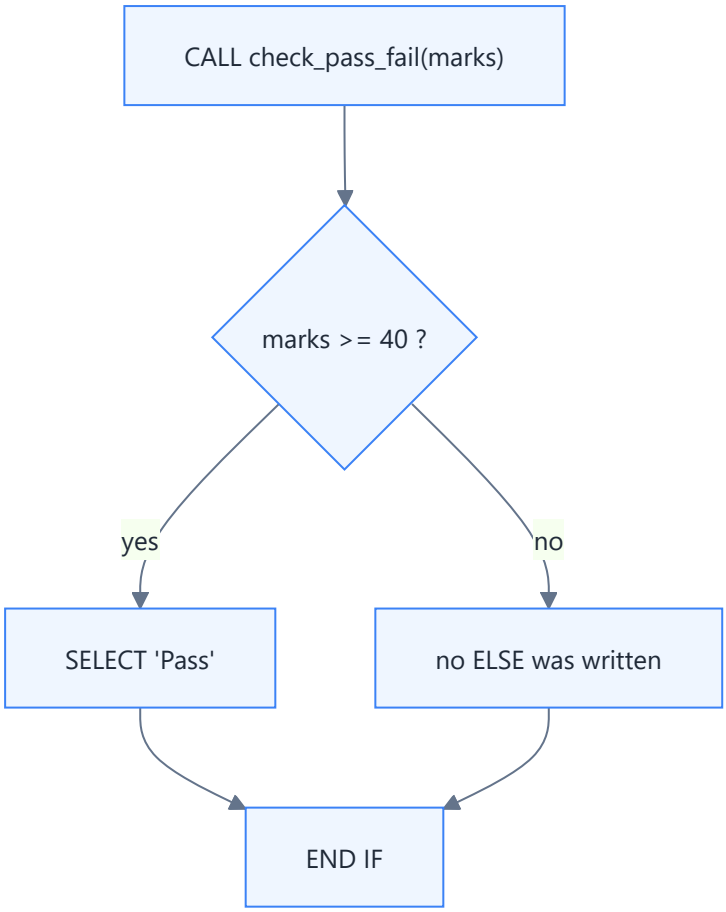
Class built a pass check. Marks of 40 or more print `Pass`. There is no `ELSE`.

```
DELIMITER $$

CREATE PROCEDURE check_pass_fail(IN s_marks INT)
BEGIN
    IF s_marks >= 40 THEN
        SELECT 'Pass' AS result;
    END IF;
END $$

DELIMITER ;

CALL check_pass_fail(65);   -- Pass
CALL check_pass_fail(40);   -- Pass  (40 >= 40)
CALL check_pass_fail(30);   -- no result set
CALL check_pass_fail(25);   -- no result set
```



Walk `30` through it. `s_marks` is 30. `30 >= 40` is false, so the `THEN` block is skipped. Nothing else was written for the false side. The procedure still ends cleanly. The client simply has **no grid to show**. That is not an error and it is not a NULL row.

> **EXAM**
> An `IF` with no `ELSE` does not print "Fail" and does not raise an error when the condition is false. The caller sees **no result**. Class came back to this later: always write an `ELSE` when the false side should say something.

3. IF … ELSE — Even or Odd with MOD

Two outcomes, both spoken out loud:

```
IF <condition> THEN
    <statements>
ELSE
    <statements>
END IF;
```

`MOD(n, d)` is the remainder of `n` divided by `d`. Same idea as Java's `%`.

Call	Division	Remainder	Meaning
`MOD(10, 2)`	10 = 5×2 + 0	0	even
`MOD(11, 2)`	11 = 5×2 + 1	1	odd

`MOD(14, 3)`	14 = 4×3 + 2	2	class's third demo

```
DELIMITER $$

CREATE PROCEDURE check_even_odd(IN num INT)
BEGIN
    IF MOD(num, 2) = 0 THEN
        SELECT 'Even' AS result;
    ELSE
        SELECT 'Odd' AS result;
    END IF;
END $$

DELIMITER ;

CALL check_even_odd(10);   -- Even
CALL check_even_odd(11);   -- Odd
```

`11` : remainder is 1, `1 = 0` is false, the `ELSE` runs, and the client always gets a row. There is no silent path anymore.

> NOTE
>
> **Q (asked in class):** "Why is `END IF` written before `END` ?" — You opened two blocks. Close the inner one (`END IF`) before you close the procedure (`END`). Indentation does not matter to MySQL. The words do.

How many outcomes?	Shape
One interesting side, silence is acceptable	`IF` … `END IF`
Two complementary sides (even/odd, pass/fail)	`IF` … `ELSE` … `END IF`
A ladder of ranges	`IF` / `ELSEIF` / `ELSE` — next section

4. IF / ELSEIF / ELSE — `calculate_grade`

> EXAM
>
> Class stopped the room for this one: take the **student id**, look the row up, and grade it. "Focus on this stored procedure. It is an important one. You will clear your concepts in this."

The shape, before the SQL:

```
IF <condition1> THEN
    <statements>
ELSEIF <condition2> THEN
    <statements>
ELSE
    <statements>     -- the default; class said always write it
END IF;
```

`ELSEIF` is **one word**. The first condition that is true wins. Later branches are not tested.

The ladder class actually wrote

Order	Test	Result
1	`s_marks IS NULL`	a message: invalid marks, or that student was not found
2	`s_marks >= 90`	`A`
3	`s_marks >= 75`	`B`
4	`s_marks >= 60`	`C`
5	`s_marks > 40`	`D`
6	`ELSE`	`F`

NULL is first on purpose. A missing student leaves the variables NULL (no row for `SELECT … INTO` to fill). `NULL >= 90` is not true, so without the NULL test the procedure would fall through into a confusing grade.

The D test is **greater than 40**, not greater-or-equal. Class said "greater than, suppose, 40". The pass procedure in section 2 used `>= 40`. Same number, different operator.

What class actually called

They did not pass marks in. They passed ids that were already in `student`.

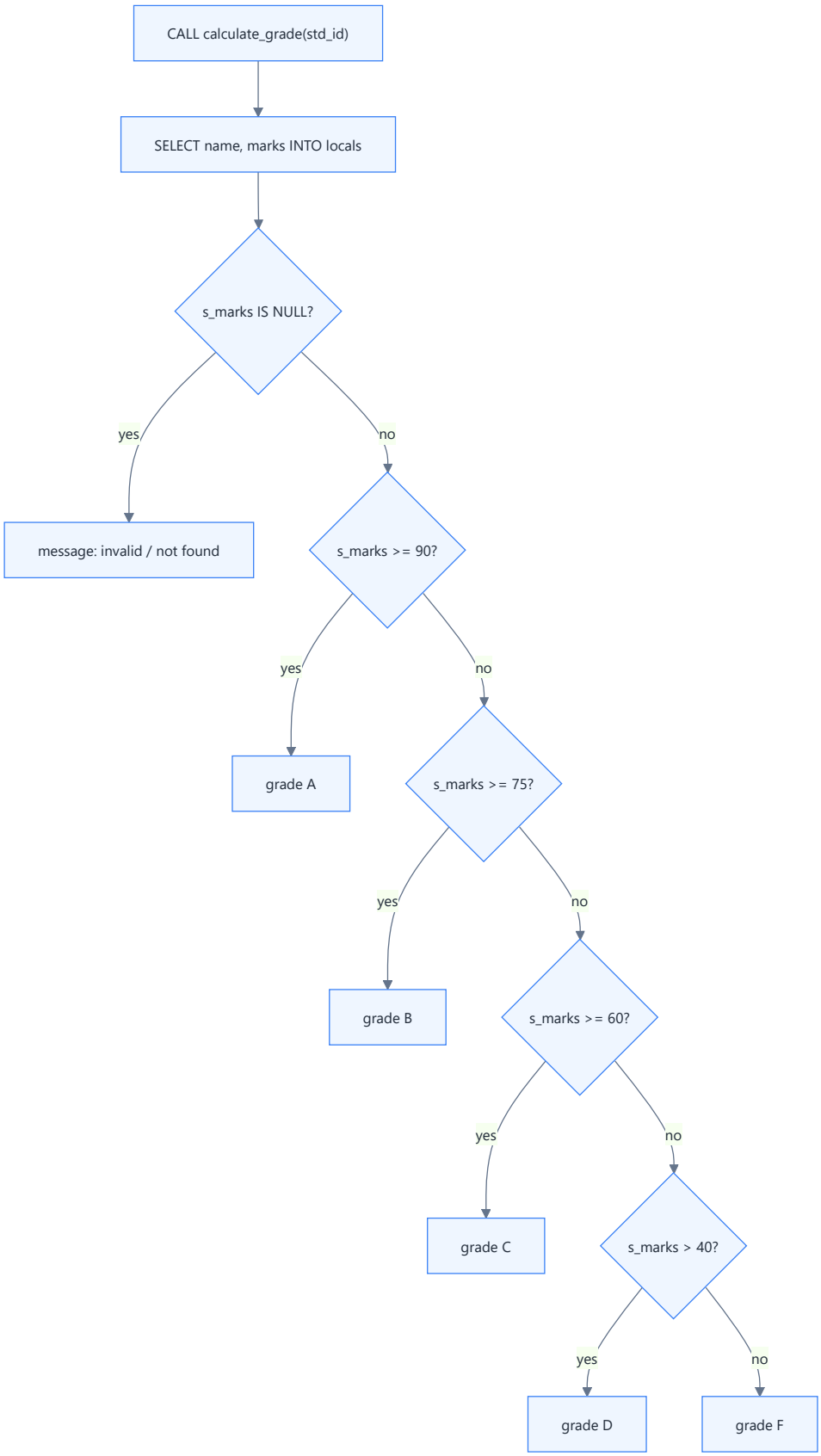
Call	What class saw
`CALL calculate_grade(101);`	grade **B**. They checked the row: marks were **85**, not above 90
`CALL calculate_grade(102);`	**Neha**, grade **A**
`CALL calculate_grade(103);`	the row was **Rahul**
`CALL calculate_grade(104);`	grade **B**
insert below, then `CALL calculate_grade(106);`	grade **F**

```
INSERT INTO student (student_id, name, department_id, marks)
VALUES (106, 'Akash', 10, 30);

CALL calculate_grade(106);   -- F, because 30 fails every test including > 40
```

Walk **85** (student 101). Not NULL. Not `>= 90`. **Is** `>= 75`, so the grade is set to `B` and the C/D/F tests are never looked at. Then the second `IF` prints the report card, because 85 is not NULL.

Walk **30** (student 106). Not NULL. Fails `>= 90`, `>= 75`, `>= 60`, and `> 40`. `ELSE` sets `F`.


CALL calculate_grade(std_id)
SELECT name, marks INTO locals
s_marks IS NULL?
yes
no
message: invalid / not found
s_marks >= 90?
yes
no
grade A
s_marks >= 75?
yes
no
grade B
s_marks >= 60?
yes
no
grade C
s_marks > 40?
yes
no
grade D
grade F

```
DELIMITER $$

CREATE PROCEDURE calculate_grade(IN std_id INT)
BEGIN
    DECLARE s_name  VARCHAR(50);
    DECLARE s_marks INT;
    DECLARE s_grade VARCHAR(10);

    SELECT name, marks
      INTO s_name, s_marks
      FROM student
     WHERE student_id = std_id;

    IF s_marks IS NULL THEN
        SELECT 'Marks are invalid or student not found' AS result;
    ELSEIF s_marks >= 90 THEN
        SET s_grade = 'A';
    ELSEIF s_marks >= 75 THEN
        SET s_grade = 'B';
    ELSEIF s_marks >= 60 THEN
        SET s_grade = 'C';
    ELSEIF s_marks > 40 THEN
        SET s_grade = 'D';
    ELSE
        SET s_grade = 'F';
    END IF;

    -- report card only when a real marks value was found
    IF s_marks IS NOT NULL THEN
        SELECT std_id  AS student_id,
               s_name  AS student_name,
               s_marks AS marks,
               s_grade AS grade;
    END IF;
END $$

DELIMITER ;
```

`DECLARE` stays at the top of `BEGIN`, before any `SELECT`. Comments inside a procedure are `--` followed by a space. Class warned that `--` jammed against the next character is not treated as a comment.

CLASS WARNING

The first version printed nothing. `SET s_grade` had run, but there was no `SELECT` of the report card, so the client stayed blank. The fix is the second `IF`, after `END IF` of the ladder. Locals still hold their values there. You do not have to `SELECT` inside every `ELSEIF`.

`SELECT … INTO` **wants one row.** Filter by the primary key. Two matching rows is an error. Zero matching rows leaves the locals NULL, which is why the NULL branch exists.

EXAM

Read the operator. Grade D in this procedure is `s_marks > 40`, so **exactly 40 is F**. The morning pass check was `s_marks >= 40`, so **exactly 40 passes**. And if you put `>= 40` *above* `>= 90` in the ladder, a student with 95 would get D, because the first true test wins.

5. CASE — Simple versus Searched

`CASE` is the other way to pick one outcome. Class reached for it because **there was no `department` table** (`SELECT * FROM department` failed) while `student` still had a `department_id`. They wanted a readable name for 10, 20, and 30.

Someone asked if this is like Java `switch`. For the first form, yes: it compares **one expression to exact values**.

Simple CASE — exact match

`department_id`	Name class printed
10	Computer Science
20	Information Technology
30	Electronics
anything else	Unknown department

```
DELIMITER $$

CREATE PROCEDURE show_department_name(IN dept_id INT)
BEGIN
    CASE dept_id
        WHEN 10 THEN
            SELECT 'Computer Science' AS department_name;
        WHEN 20 THEN
            SELECT 'Information Technology' AS department_name;
        WHEN 30 THEN
            SELECT 'Electronics' AS department_name;
        ELSE
            SELECT 'Unknown department' AS department_name;
    END CASE;
END $$

DELIMITER ;

CALL show_department_name(10);   -- Computer Science
CALL show_department_name(40);   -- Unknown department
```

You cannot write `WHEN >= 90` in this form. The value after `CASE` is compared for **equality** only. A grade of "exactly 90" would match `WHEN 90`. A grade of 91 would not.

Searched CASE — each WHEN is a condition

Leave the expression off the `CASE` line. Each `WHEN` is a full test, the same job as `ELSEIF`. This is the form that can say `>=`.

```
CASE
    WHEN s_marks >= 90 THEN
        SELECT 'A' AS grade;
    WHEN s_marks >= 75 THEN
        SELECT 'B' AS grade;
    WHEN s_marks >= 60 THEN
        SELECT 'C' AS grade;
    WHEN s_marks > 40 THEN
        SELECT 'D' AS grade;
    ELSE
        SELECT 'F' AS grade;
END CASE;
```

Drop that block into `calculate_grade` in place of the `IF` ladder if you want the same result written the other way. You still need `DELIMITER`, `CREATE PROCEDURE`, the `SELECT … INTO`, and **`END CASE`** (not `END IF`).

	Simple CASE	Searched CASE
Header	`CASE dept_id`	`CASE` alone
`WHEN`	a value: `WHEN 10`	a test: `WHEN s_marks >= 90`
Use it for	codes, status flags, department ids	ranges
Java cousin	`switch`	`if` / `else if`
Closer	`END CASE`	`END CASE`

CLASS WARNING

`CASE s_marks WHEN s_marks >= 90` is illegal. Either put the variable after `CASE` and use `WHEN 90` (exact 90 only), or omit the variable and write `WHEN s_marks >= 90`.

NOTE

Q (asked in class): "Is a procedure the same as a function?" — It does similar work, and the difference is the next class. Today stays on procedures. If you have two `CASE` blocks in one procedure, each one still needs its own `END CASE`.

Part B — Loops

In this part: `LOOP` + `LEAVE` · `WHILE` · `REPEAT … UNTIL` · `ITERATE` · a nested `WHILE`.

A loop repeats a set of statements until an end condition stops it. Class's picture was the same one from Java: you do not write ten `SELECT`s to print 1 through 10. MySQL stored programs have no `for`. They have the three forms below.

Day 6 already used `WHILE` to insert a lakh of rows. Today's loops are tiny so you can **see** each pass. Every `SELECT` inside the loop comes back as its **own** result grid.

Form	When the test runs	Java cousin
`LOOP` + `LEAVE`	wherever you put the `IF`	`while (true) { if (done) break; }`
`WHILE … DO`	**before** the body	`while (cond) { … }`
`REPEAT … UNTIL`	**after** the body	`do { … } while (!cond)`

`LEAVE` is break. `ITERATE` is continue. Both need the loop's **label**.

> **CLASS WARNING**
>
> Forget `SET i = i + 1` and the loop never ends. On a procedure that inserts rows, that fills the table until you kill the session. The end condition has to become false (or true, for `UNTIL`) on its own.

6. LOOP with LEAVE — Print 1 to 5

A bare `LOOP` never stops. You name it, and when the exit test is true you `LEAVE` that name.

```
label_name: LOOP
    statements
    IF exit_condition THEN
        LEAVE label_name;
    END IF;
END LOOP label_name;
```

The label is your name. Class used `number_loop`. It is not a keyword. The word `LOOP` after the colon **is** the keyword.

```
DELIMITER $$

CREATE PROCEDURE print_numbers_in_loop()
BEGIN
    DECLARE i INT DEFAULT 1;

    number_loop: LOOP
        SELECT i AS number;
        SET i = i + 1;
        IF i > 5 THEN
            LEAVE number_loop;
        END IF;
    END LOOP number_loop;
END $$

DELIMITER ;

CALL print_numbers_in_loop();
```

`CALL` produced five grids: 1, then 2, then 3, then 4, then 5.

Arriving at the loop	`SELECT` prints	after `SET i = i + 1`	`i > 5` ?
1	1	2	no, loop again
2	2	3	no
3	3	4	no
4	4	5	no
5	5	6	yes → `LEAVE`

The test sits **after** the print and the increment, so 5 is printed and 6 is not. `DEFAULT 0` would have printed 0 first. The default is the first value `SELECT i` sees.

> NOTE
>
> **Q (asked in class):** "Is `LOOP` after `number_loop:` a keyword?" — Yes. The name is yours; `LOOP` is the statement. And "what does `LEAVE` mean?" — leave the loop now. It is the break. It runs only when the `IF` is true.

7. WHILE — Test Before the Body

```
WHILE <condition> DO
    <statements>
END WHILE;
```

The condition is checked **before** the body. If it is already false, the body runs zero times. `END WHILE` is required, same idea as `END IF`.

Class first wrote `WHILE i < 5` with `i` starting at 1. Walk that:

- 1 < 5, print 1, i = 2

- 2 < 5, print 2, i = 3
- 3 < 5, print 3, i = 4
- 4 < 5, print 4, i = 5
- 5 < 5 is **false** — **5 is never printed**

That is the whole difference from section 6, where the test sat after the print. They changed the condition to `<=`.

```
DELIMITER $$

CREATE PROCEDURE print_numbers_while()
BEGIN
    DECLARE i INT DEFAULT 1;

    WHILE i <= 5 DO
        SELECT i AS number;
        SET i = i + 1;
    END WHILE;
END $$

DELIMITER ;

CALL print_numbers_while();   -- 1, 2, 3, 4, 5
```

i at the test	`i <= 5`	prints	i afterwards
1	true	1	2
2	true	2	3
3	true	3	4
4	true	4	5
5	true	5	6
6	false	stop	

`DEFAULT 0` starts at 0. `DEFAULT 2` starts at 2. The default is not a suggestion; it is the first value the test sees.

> **EXAM**
> `WHILE` is a **pre-test**. `i < 5` and `i <= 5` are not interchangeable. If the body must run at least once even when the test starts out false, that is `REPEAT`, not `WHILE`.

8. REPEAT ... UNTIL — Body First, Then the Test

`REPEAT` runs the body, **then** asks whether to stop. It keeps going **until the UNTIL expression becomes true**. So the body runs at least once.

```
REPEAT
    <statements>
UNTIL <exit condition>
END REPEAT;
```

Read `UNTIL i > 5` as: keep going until i has become greater than 5. While that test is still false, do it again.

```
DELIMITER $$

CREATE PROCEDURE print_numbers_repeat()
BEGIN
    DECLARE i INT DEFAULT 1;

    REPEAT
        SELECT i AS num;
        SET i = i + 1;
    UNTIL i > 5
    END REPEAT;
END $$

DELIMITER ;

CALL print_numbers_repeat();   -- 1, 2, 3, 4, 5
```

1. Print 1, i = 2. Is 2 > 5? No.
2. Print 2, i = 3. No.
3. Print 3, i = 4. No.
4. Print 4, i = 5. No.
5. Print 5, i = 6. **6 > 5 is true** → stop.

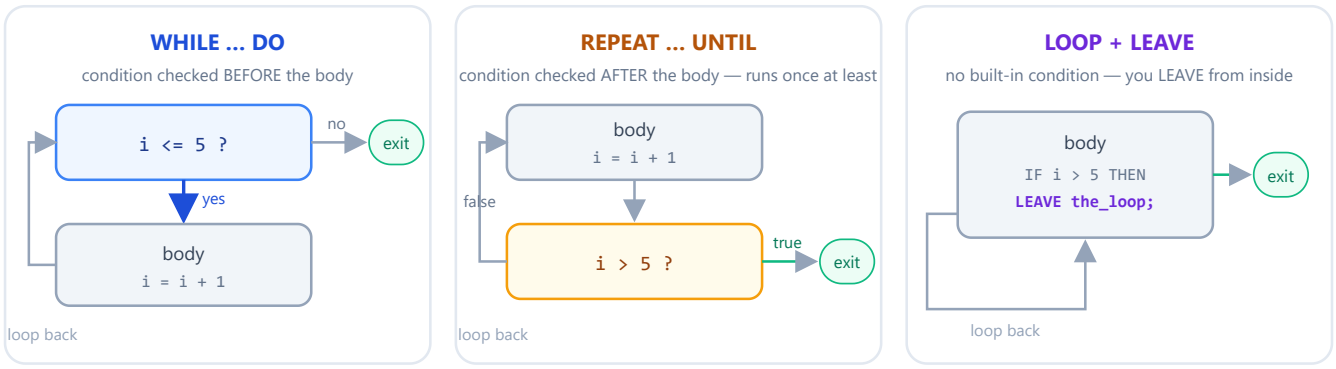
Class then changed the line to `UNTIL i >= 5`. After printing 4, i becomes 5, and `5 >= 5` is already true, so the output **stops at 4**. They ran it and confirmed that.

> NOTE
>
> **Q (asked in class):** "If it were `>=`, would it be the same?" — No. `UNTIL i > 5` prints through 5. `UNTIL i >= 5` prints through 4. The body has already incremented before the test.

There is no `END UNTIL`. The closer is `END REPEAT`.

9. Where the Condition Sits

Same job — print 1 through 5, starting at 1 — three different tests, because the test happens at a different moment.



WHILE tests before the body, REPEAT tests after it, LOOP tests only where you write LEAVE

Loop	Condition that prints 1..5 when i starts at 1
`WHILE`	`WHILE i <= 5 DO` — test **before** the print
`LOOP` + `LEAVE`	`IF i > 5 THEN LEAVE` **after** print and `SET i = i + 1`
`REPEAT`	`UNTIL i > 5` **after** print and `SET i = i + 1`

> **EXAM**
> `WHILE` continues **while** its expression is true, and it checks first. `REPEAT` continues **until** its expression is true, and it checks after the body. Opposite sense of the boolean, and opposite time of the test.

10. ITERATE — Skip One Pass

Keyword	Java	What it does
`LEAVE label`	`break`	exit that labeled loop now
`ITERATE label`	`continue`	skip the **rest of this pass** and go to the next test of that loop

Class printed numbers with `WHILE i < 10`, but skipped 5. The increment is the **first** statement in the body, and the `SELECT` is **after** the skip test.

```
DELIMITER $$

CREATE PROCEDURE print_numbers_skip_5()
BEGIN
    DECLARE i INT DEFAULT 1;

    number_loop: WHILE i < 10 DO
        SET i = i + 1;
        IF i = 5 THEN
            ITERATE number_loop;
        END IF;
        SELECT i AS number;
    END WHILE number_loop;
END $$

DELIMITER ;

CALL print_numbers_skip_5();
-- grids: 2, 3, 4, 6, 7, 8, 9, 10
-- 5 is skipped; 1 never appears because the first thing the body does is add 1
```

Why 10 still prints with `WHILE i < 10`: when i is 9 the test is true, the body sets i to 10, 10 is not 5, so 10 is selected. The next test is `10 < 10`, which is false.

Why 1 is missing: i starts at 1, the test `1 < 10` passes, then `SET` turns it into 2 **before** the `SELECT`. A student asked what happens if `SET` is moved to after the `SELECT`. Then 1 would print. If the `SELECT` is written *before* the `IF i = 5` test, 5 prints too, because `ITERATE` only skips statements that have not run yet.

To skip both 3 and 5, the test is `IF i = 3 OR i = 5`. `AND` can never be true for a single value of i.

> **CLASS WARNING**
>
> `ITERATE` needs the label (`ITERATE number_loop`). A second loop would be closed with its own name: `END WHILE number_loop1` . A signed `INT` default **may** be negative. `INT UNSIGNED` may not.

> **NOTE**
>
> **Q (asked in class):** a student pointed at the line numbers and said moving the increment below the `SELECT` would print from 1. That is right. Order inside the body is the output.

11. Nested WHILE — a 2 by 3 Grid

A nested loop is a loop inside a loop. For **each** value of the outer counter, the inner counter runs its whole range. Class described it as the coordinates of a 2×3 matrix: rows 1..2, columns 1..3.

The detail that makes the second row exist: **set j back to 1 inside the outer loop**. If you only declare j and never reset it, j is already past 3 after the first row and the inner loop never runs again.

```
DELIMITER $$

CREATE PROCEDURE nested_loop_demo()
BEGIN
    DECLARE i INT DEFAULT 1;
    DECLARE j INT;

    outer_loop: WHILE i <= 2 DO
        SET j = 1;
        inner_loop: WHILE j <= 3 DO
            SELECT i AS r_no, j AS c_no;
            SET j = j + 1;
        END WHILE inner_loop;
        SET i = i + 1;
    END WHILE outer_loop;
END $$

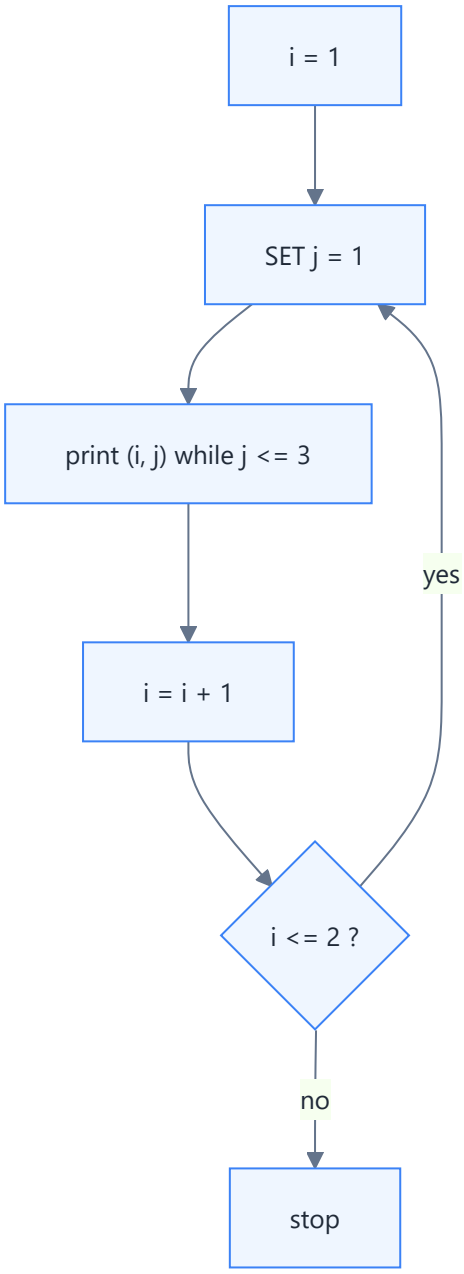
DELIMITER ;

CALL nested_loop_demo();
```

Six grids:

r_no	c_no
1	1
1	2
1	3
2	1
2	2
2	3

First row only: i is 1, which is `<= 2`, so j is set to 1. Inner prints (1,1), (1,2), (1,3), then j becomes 4 and `4 <= 3` fails. Only then does i become 2, and j is set to 1 **again**. When i becomes 3, `3 <= 2` fails and the outer loop ends.



CLASS WARNING

Class first aliased the columns as `row_number`. MySQL treated that as the **window function** `ROW_NUMBER` and the procedure failed. Do not use `ROW_NUMBER`, `RANK`, or `DENSE_RANK` as aliases. A stray space in the label also broke one paste. The aliases above (`r_no`, `c_no`) are the safe version of the same demo.

A procedure is allowed to touch **more than one table**, including joins. The grid above is only a teaching loop. The real use class described is one `CALL` that inserts a registration into `student` and into a related table, so the same statements are not retyped every time.

Nested block — the example class said would be in the notes

A nested **block** is `BEGIN … END` inside another `BEGIN … END`. Variables declared in the inner block die when that block ends. You can also label a `BEGIN` and `LEAVE` it, the same way you leave a loop.

```
DELIMITER $$

CREATE PROCEDURE nested_block_demo()
BEGIN
    DECLARE x INT DEFAULT 10;
    BEGIN
        DECLARE y INT DEFAULT 20;   -- y exists only inside this inner BEGIN
        SELECT x AS outer_var, y AS inner_var;
    END;
    SELECT x AS still_here;
END $$

DELIMITER ;
```

Part C — Built-in Functions

In this part: string functions · numeric functions · date and time functions.

12. Why This Part Is on the Exam

Built-in functions are functions MySQL already has. You call them. You do not `CREATE FUNCTION` for them. Custom stored functions are the next class.

> **EXAM**
> Class said this part is easy and that it **will** be on the module exam: at least **two or three** questions. Learn the return value, the argument order, and the one trap on each function (1-based positions, `ROUND` versus `TRUNCATE`, `%Y` versus `%y`).

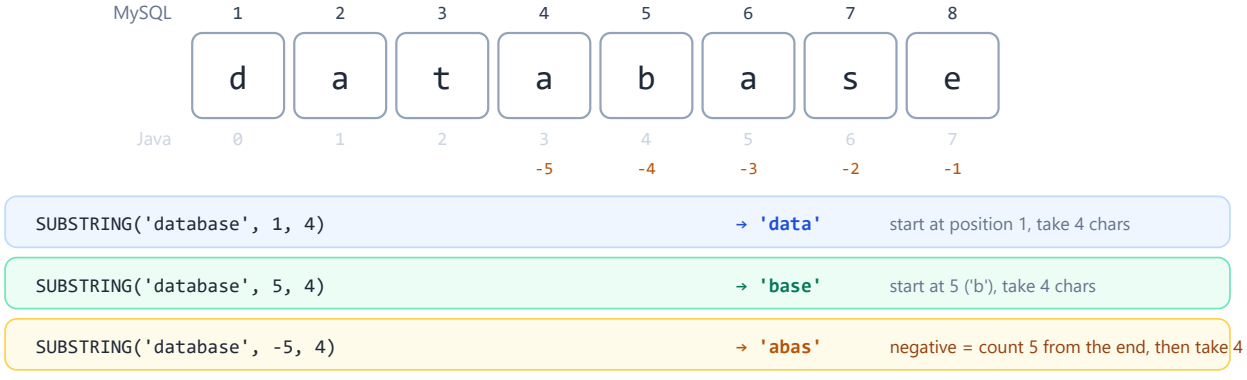
Three families, in the order class taught them: **string**, then **numeric**, then **date/time**.

A `SELECT LOWER(name)` **shows** a value. It does not change the table. To store the change you need `UPDATE … SET col = function(col)`.

> **TIP**
> **Assignment from class, before the function break.** Write a procedure that registers a new student. Inputs: id, name, marks. One `INSERT` into `student`. The finished version is in the day wrap.

13. String Functions

MySQL string positions start at **1**. Java starts at **0**. That single fact is most of the substring MCQs.



MySQL positions on 'database' start at 1; a negative start counts back from the end

LOWER and UPPER

```
SELECT LOWER('Hello World');   -- hello world
SELECT UPPER('Hello World');   -- HELLO WORLD

SELECT student_id, UPPER(name) AS name_upper FROM student;
SELECT student_id, LOWER(name) AS name_lower FROM student;
```

Class updated a couple of names (Amit Patil, Tiya Bapat, and later Akash) so the case change was obvious, then ran both queries. The stored rows stay as they were until an `UPDATE`.

SUBSTRING — start position, then how many characters

```
SELECT SUBSTRING('database', 1, 4);    -- data
SELECT SUBSTRING('database', 5, 4);    -- base
SELECT SUBSTRING('database', -5, 4);   -- abas
```

Argument 2 is where to start. Argument 3 is the **length**, not the end index. A negative start counts from the end: the last five characters of `database` are `a b a s e`, and taking four of them gives `abas`. Class tried this live and had to correct an earlier positive-5 note on the board.

A space inside the literal counts, and it also makes the parser fail if it lands in the wrong place. You can pass a column: `SUBSTRING(name, 1, 4)`.

LEFT and RIGHT

```
SELECT LEFT('database', 4);    -- data
SELECT RIGHT('database', 4);   -- base
```

`LEFT(s, n)` is the first n characters. `RIGHT(s, n)` is the last n. Same results as the two positive `SUBSTRING` calls above.

TRIM — spaces on both ends

Class padded a name with spaces and showed that `SELECT name` still had them, while `TRIM(name)` displayed a clean string.

```
UPDATE student
SET name = '  Amit Patil  '
WHERE student_id = 101;

SELECT TRIM(name) AS cleaned
FROM student
WHERE student_id = 101;
-- Amit Patil, spaces gone on both ends
```

> NOTE
>
> **Q (asked in class):** "Does TRIM remove spaces from both ends?" — Yes, leading and trailing. And the `SELECT` does **not** write the cleaned value back. `SELECT *` afterwards still shows the spaces. To keep the change:
>
> ```
> UPDATE student
> SET name = TRIM(name)
> WHERE student_id = 101;
> ```
>
> Omit the `WHERE` and every name in the table is trimmed. That is often what you want after messy data entry.

REPLACE — every occurrence

```
SELECT REPLACE('I like Java', 'Java', 'MySQL');
-- I like MySQL

SELECT REPLACE('I like Java and Java', 'Java', 'MySQL');
-- I like MySQL and MySQL
```

A student asked whether a repeated word is replaced twice. Class ran it: **every** occurrence, not just the first. The other demo was `color` → `colour`. The practical story was an email: replace `@gmail.com` with `@company.com`.

`SELECT REPLACE(name, 'Shinde', 'Chaudhary') FROM student` showed Neha's new surname **in the result only**. `SELECT *` still had Shinde.

```
UPDATE student
SET name = REPLACE(name, 'Shinde', 'Chaudhary')
WHERE student_id = 102;
```

> **CLASS WARNING**
> Do not write `UPDATE student SET name = REPLACE(...) FROM student`. MySQL error: you can't specify the target table that way. There is no `FROM`. The column you are changing is already `name` on `student`. Without a `WHERE`, every row is updated — which is correct when the same bad spelling is in many names, and wrong when you only meant one student.

INSTR — first position, or 0

`INSTR(haystack, needle)` returns the **1-based position of the first match**. If the needle is absent, the result is **0**, not NULL.

```
SELECT INSTR('aniket@gmail.com', '@');     -- 7
-- a=1 n=2 i=3 k=4 e=5 t=6 @=7

SELECT INSTR('aniket@gmail.com', 'xyz');   -- 0
```

On the table, searching names for `t`: the first `t` wins if the letter repeats. A name with no `t` returns 0. Amit's `t` looked like position 7 instead of 4 because the **leading spaces** from the TRIM demo were still in the column. Spaces count.

```
SELECT name, INSTR(name, 't') AS t_pos
FROM student;
```

LPAD and RPAD

```
SELECT LPAD('123', 5, '0');   -- 00123
SELECT RPAD('123', 5, '0');   -- 12300
```

The third argument is the pad character. The second is the **total length**, not "how many pads". `'123'` is a string here. As a number, the zeros would not exist to pad.

Class's example: a mobile number that should grow a country code, or leading zeros, out to a fixed width. The same `UPDATE SET col = LPAD(col, …)` pattern used for `REPLACE` stores it.

REVERSE

```
SELECT REVERSE('MySQL');   -- LQSyM
```

String functions — the card

Function	Returns	Trap from class
`LOWER` / `UPPER`	case change	`SELECT` does not update the row
`SUBSTRING(s, pos, len)`	a slice	pos is **1-based**; negative pos counts from the end; third arg is length
`LEFT` / `RIGHT`	n characters from that end	
`TRIM`	both ends' spaces removed	does not persist until `UPDATE`
`REPLACE`	**all** occurrences	`UPDATE … SET col = REPLACE(col, …)` with no `FROM`
`INSTR`	first position, or **0**	leading spaces shift the number
`LPAD` / `RPAD`	string padded out to a length	pad character is the third argument
`REVERSE`	characters reversed	

14. Numeric Functions

ROUND versus TRUNCATE

Both take a number and a count of decimal places. `ROUND` looks at the **next** digit. If that digit is 5 or more, the last kept digit goes up. If it is 4 or less, it stays. `TRUNCATE` **cuts**. It does not look at the leftover.

```
SELECT ROUND(20.867, 2);       -- 20.87   (the 7 pushes 6 up to 7)
SELECT ROUND(20.8641, 2);      -- 20.86   (the 4 does not)
SELECT TRUNCATE(53.34589, 3);  -- 53.345  (89 is thrown away, not rounded to 53.346)
```

Class used a percentage story for `ROUND`: a mark like 92.85678 that has to be shown to two places. They also showed that a digit below 5 (they tried a 41 in that position) does **not** round up.

> **EXAM**
> `ROUND(20.867, 2)` is 20.87. `TRUNCATE` of a number that *would* have rounded is still the chopped value. If the question says "remove extra decimals without rounding", the function is `TRUNCATE`.

You do not need a column constraint to call these. `marks DECIMAL(5,2)` is a **storage** choice. `SELECT ROUND(marks, 2)` is a **query-time** choice. Class said there is no need to push the rounding into a constraint when the function already does it.

CEIL and FLOOR

`CEIL` (also spelled `CEILING`) returns the smallest integer that is **greater than or equal** to the value. `FLOOR` returns the largest integer that is **less than or equal** to it.

```
SELECT CEIL(45.2);    -- 46
SELECT FLOOR(45.8);   -- 45
SELECT CEIL(45);      -- 45   already an integer
SELECT FLOOR(45);     -- 45
```

Class's memory line: 47-point-something goes to 48 with `CEIL` and stays 47 with `FLOOR`. If the value is already a whole number, both return that number, so you would not call them.

MOD, ABS, POWER, SQRT, RAND

```
SELECT MOD(10, 2);    -- 0
SELECT MOD(11, 2);    -- 1
SELECT MOD(14, 3);    -- 2     class ran this one

SELECT ABS(-28);      -- 28
SELECT POWER(2, 3);   -- 8     2 to the power 3
SELECT POWER(3, 3);   -- 27
SELECT SQRT(25);      -- 5
SELECT SQRT(20);      -- a decimal; it does not have to be a perfect square
SELECT SQRT(-25);     -- NULL

SELECT RAND();        -- a float from 0 up to but not including 1
SELECT RAND();        -- a different float
```

`MOD` is the same remainder already used for even/odd. `ABS` turns a negative into a positive and leaves a positive alone. `SQRT` of a negative is **NULL**, not an error row — class tried −25 and got NULL. `RAND()` is not a dice roll to an integer; each call is a new float.

Function	Returns
`ROUND(x, d)`	x rounded to d decimals
`TRUNCATE(x, d)`	x cut to d decimals
`CEIL` / `CEILING`	smallest integer ≥ x
`FLOOR`	largest integer ≤ x
`MOD(a, b)`	remainder
`ABS(x)`	magnitude, never negative
`POWER(a, b)`	a to the power b
`SQRT(x)`	square root; negative → NULL
`RAND()`	a new float in [0, 1)

15. Date and Time Functions

Class drew the three types as three different widths of "now":

Type	What it holds	Shape they wrote
`DATE`	the calendar day	`2026-10-15`
`TIME`	clock only	`13:28:55`
`DATETIME`	both	`2026-10-15 13:29:40`

NOW, CURRENT_DATE, CURRENT_TIME

```
SELECT NOW();             -- date and time of this instant
SELECT CURRENT_DATE();    -- date only
SELECT CURRENT_TIME();    -- time only
```

`NOW()` is not "the date". It is date **plus** time. You can nest them. Class tried it when a student asked:

```
SELECT YEAR(NOW()), MONTH(NOW()), DAY(NOW());
```

YEAR, MONTH, DAY

```
SELECT YEAR('2026-10-15');    -- 2026
SELECT MONTH('2026-10-15');   -- 10
SELECT DAY('2026-10-15');     -- 15
```

On the lecture day, `YEAR(NOW())`, `MONTH(NOW())`, and `DAY(NOW())` returned 2026, 10, and 15.

DATE_FORMAT — a date becomes the string you want

The second argument is a pattern of `%` codes, in quotes. Forgetting the quotes was the first error.

```
SELECT DATE_FORMAT('2026-10-15', '%d-%m-%Y');   -- 15-10-2026
SELECT DATE_FORMAT('2026-10-15', '%m/%d/%Y');   -- 10/15/2026
SELECT DATE_FORMAT('2026-10-15', '%M %d, %Y');  -- October 15, 2026
```

Code	Meaning	For 15 Oct 2026
`%Y`	4-digit year	2026
`%y`	2-digit year	26
`%m`	month **number**	10
`%M`	month **name**	October
`%d`	day of month	15
`%H`	hour	
`%i`	minutes	

`%s`	seconds	
`%W`	weekday **name**	Thursday, on that lecture day

Class named `%H`, `%i`, and `%s` for a datetime that includes a clock, and told the room to look up the code that prints "Thursday". That code is `%W`.

STR_TO_DATE — a string becomes a date

The reverse of `DATE_FORMAT`. You must say which piece of the text is the day, the month, and the year.

```
SELECT STR_TO_DATE('15-10-2026', '%d-%m-%Y');   -- 2026-10-15
```

Class first used `%y` against the four-digit year `2026` and the year came out wrong (they saw a 2020). `%y` consumes **two** digits. `%Y` consumes **four**. Switching the pattern to `%Y` fixed it. The month and the day had been right the whole time, which is why the bug was easy to miss.

EXAM

`%Y` versus `%y`, and `%M` versus `%m`, are the pair to not swap. `NOW()` includes the time. `CURRENT_DATE()` does not. `DATE_FORMAT` goes date → string. `STR_TO_DATE` goes string → date. The `%` letters mean the same family of things in both.

CLASS WARNING

Point `%y` at `2026` and you do not get 2026. Point `%Y` at a two-digit year and you are guessing. The pattern has to match the text.

16. Day Wrap — Rapid Revision and the Assignment

The assignment

A new student registers. The procedure takes the id, the name, and the marks, and inserts one row. Add `department_id` to the parameter list if your table requires it.

```
DELIMITER $$

CREATE PROCEDURE register_student(
    IN p_id    INT,
    IN p_name  VARCHAR(50),
    IN p_marks INT
)
BEGIN
    INSERT INTO student (student_id, name, marks)
    VALUES (p_id, p_name, p_marks);
END $$

DELIMITER ;

CALL register_student(107, 'New Student', 55);
SELECT * FROM student WHERE student_id = 107;
```

Conditions and loops

Topic	One-liner
`END IF` / `END CASE` / `END WHILE` / `END REPEAT` / `END LOOP`	close the block you opened; `END` alone closes the procedure
Silent `IF`	no `ELSE`, false condition → no result set
`ELSEIF`	one word; first true test wins; write ranges high to low
Grade D in class's ladder	`s_marks > 40`, so 40 is F. The pass check was `>= 40`
Simple vs searched `CASE`	exact value vs a full condition; both end with `END CASE`
`LEAVE` / `ITERATE`	break / continue; both need the label
`WHILE` vs `REPEAT`	test before the body vs after; `REPEAT` runs at least once
`WHILE i < 5`	prints 1..4 when i starts at 1
`UNTIL i >= 5`	prints 1..4; `UNTIL i > 5` prints 1..5
Nested loop	reset the inner counter on every outer pass
Alias trap	do not name a column `row_number`

Built-ins

Topic	One-liner
String index	MySQL starts at **1**
`INSTR` miss	**0**, not NULL; first match only
`REPLACE` / `TRIM` in a `SELECT`	display only, until an `UPDATE`
`ROUND` vs `TRUNCATE`	looks at the next digit vs cuts
`SQRT` of a negative	NULL
`RAND()`	a new float in [0, 1)
Date codes	`%Y` four digits, `%y` two, `%M` month name, `%m` month number

EXAM

Straight from class:

- Two or three CCEE questions from the built-in list.
- Always write the `ELSE`. It is the default when the input is unexpected.
- `DELIMITER ;` with the space, every time, or the next statement never gets sent.
- `END IF` before `END`. They close different blocks.

Practice (lab order)

1. `CALL check_pass_fail(30)` and confirm the empty result. Then add an `ELSE` that says Fail.

2. `CALL check_even_odd(10)` and `CALL check_even_odd(11)`.
3. Dry-run `calculate_grade` for 85, for a mark above 90, for 30, and for an id that is not in the table. Then `CALL` it.
4. Rewrite the ladder as a searched `CASE`. Write one simple `CASE` for department ids 10, 20, and 30.
5. Print 1..5 three ways. Then change `WHILE` to `i < 5` and `UNTIL` to `i >= 5`, and predict the grids before you run them.
6. `print_numbers_skip_5`. Move `SET` to after `SELECT` and explain why 1 appears.
7. Run the 2×3 nested loop. Remove `SET j = 1` once and look at the broken second row.
8. `SUBSTRING('database', 1, 4)`, `(5, 4)`, and `(-5, 4)`. `INSTR('aniket@gmail.com', '@')`.
9. `ROUND(20.867, 2)` next to `TRUNCATE(53.34589, 3)`. `SQRT(-25)`.
10. `DATE_FORMAT` with `%Y` and with `%y`, and `STR_TO_DATE('15-10-2026', '%d-%m-%Y')`.

> **Next session:** stored functions — how they differ from the procedures written today.
>
> Coverage: DBT Day 7 lecture (15 Oct 2026), from the class transcript. Part A = IF and CASE · Part B = loops · Part C = built-in functions.